IEEE-CIS Fraud Detection: Predictive Pipeline

1. Project Overview

This project addresses the high-stakes challenge of Financial Fraud Detection using the IEEE-CIS dataset. The goal was to build a robust machine learning pipeline capable of identifying fraudulent transactions while navigating extreme class imbalance and high-dimensional data.

- Objective: Predict the probability of fraud (isFraud) for large-scale transactions.
- Primary Metric: ROC-AUC (Receiver Operating Characteristic - Area Under Curve).
- Final Achievement: Accomplished a 0.89 CV ROC-AUC, successfully navigating complex data leakage and memory constraints.

2. Tech Stack

Category	**Tools & Libraries**
Language	Python 3.x
Data Manipulation	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Modeling	**LightGBM** (Gradient Boosting), Scikit-learn
Validation Strategy	Stratified K-Fold Cross-Validation (OOF Predictions)
Optimization	Memory reduction techniques (downcasting), Hyperparameter Tuning

3. Key Feature Engineering

Feature engineering was the primary driver of model performance. The following features were synthesized to capture fraudulent behavior:

- Time-Based: TransactionHour, TransactionDay to find high-risk time windows.
- User Identity (Aggregation): Transaction counts and average amounts per cardID (card1_count, card1_avg_amt).
- Behavioral Delta: time_diff (time since the last transaction for a specific user).
- Statistical Normalization: Z-score normalization of transaction amounts to identify outliers.
- Interaction Features: Concatenated identifiers (e.g., card_addr) to create unique user fingerprints.

4. The Challenge: Debugging & Lessons Learned

Building a model is easy; building a *correct* one is hard. This project involved deep troubleshooting:

Critical Hurdles Overcome:

1. Categorical Inconsistency: A major performance drop (0.89 → 0.45) occurred due to inconsistent label encoding between Train and Test. This taught me that consistency is more important than complexity.
2. Memory Management: Handling millions of rows led to MemoryError. I implemented data type optimization and simplified feature matrices to maintain a stable environment.
3. Data Leakage Prevention: Identified that product_risk features must be calculated strictly within CV folds to prevent "looking into the future."

5. Results & Performance

- Baseline Model: 0.76 ROC-AUC
- Feature Engineered Model: 0.86 ROC-AUC
- Final Optimized Model: 0.89 ROC-AUC (CV)
- Kaggle Public Leaderboard: 0.80 ROC-AUC

Note: The gap between CV and Leaderboard scores highlighted the importance of handling distribution shifts between training and unseen test data.

6. Professional Competencies Gained

- End-to-End Pipeline Design: From raw data cleaning to final submission file generation.
- Robust Validation: Expertise in Stratified K-Fold to ensure model generalization.
- Data Integrity: Deep understanding of data leakage and how to avoid it in production-like settings.
- Resource Optimization: Managing high-dimensional datasets under hardware constraints.